React Native API Calls Tutorial

A comprehensive guide to making API calls in React Native, progressing from callbacks to modern async/await patterns.

Table of Contents

1. Installation Guide
2. Introduction
3. API Calls with Callbacks
4. API Calls with Promises
5. Modern Way: Async/Await
6. Error Handling with Try/Catch
7. Best Practices
8. Real Examples from This Project

Installation Guide

How to Run the Project

Follow these steps to set up and run the Lab 6 project:

Step 1: Download the Repository

Download the project from GitHub:

Repository URL: https://github.com/neonbranch/CPSC-357-Lab

Step 2: Extract and Navigate

1. Extract the downloaded files
2. Navigate to the `lab6` folder

```
cd lab6
```

Step 3: Install Dependencies

Install all required npm packages:

```
npm install
```

Required packages for this lab:

- `@react-navigation/native`

- `@react-navigation/native-stack`
- `@react-navigation/bottom-tabs`
- `@react-navigation/drawer`
- `expo-notifications`
- `expo-device`
- `expo-constants`
- `react-native-safe-area-context`
- `react-native-screens`
- `react-native-gesture-handler`

Step 4: Start the Backend API Server

Important: This project requires a backend API server to be running. Make sure you have the backend server running before starting the Expo app.

1. Navigate to your backend API directory (if separate from this project)
2. Start the backend server (typically on port 3000):

```
# Example: If your backend is in a separate directory
cd ../backend-api
npm install
npm start
```

Backend API Requirements:

- The API should be running on `http://localhost:3000` (or your configured IP address)
- Update the `API_BASE_URL` in `constants/api.js` to match your backend server address
- For Android Emulator, use: `http://10.0.2.2:3000/api`
- For iOS Simulator, use: `http://localhost:3000/api`
- For Physical Device, use your computer's IP: `http://YOUR_IP_ADDRESS:3000/api`

To find your IP address:

- **Windows**: Run `ipconfig` in Command Prompt and look for IPv4 Address
- **Mac/Linux**: Run `ifconfig` in Terminal and look for inet address

Step 5: Start the Expo Development Server

Run the Expo development server:

```
npx expo start
```

This will start the Metro bundler and display a QR code. You can:

- Press `a` to open on Android emulator
- Press `i` to open on iOS simulator

- Scan the QR code with Expo Go app on your physical device

Note: Make sure your device/emulator and computer are on the same network when using a physical device.

Step 6: Configure API Base URL

Before testing API calls, make sure to update the API base URL in `constants/api.js`:

```
// constants/api.js
export const API_BASE_URL = 'http://YOUR_IP_ADDRESS:3000/api';
```

Replace `YOUR_IP_ADDRESS` with your actual IP address or use the appropriate address for your development environment.

This project demonstrates:

- Making API calls with async/await
- User authentication with JWT tokens
- Fetching and displaying user profile data
- Error handling and loading states
- React Context API for state management

Introduction

When making API calls in JavaScript/React Native, you're dealing with **asynchronous operations** - operations that take time to complete (like network requests). JavaScript provides several ways to handle these:

1. **Callbacks** - The traditional way (can lead to "callback hell")
2. **Promises** - A cleaner approach
3. **Async/Await** - The modern, most readable way

This tutorial will guide you through each approach, showing how to evolve from callbacks to the modern async/await pattern.

> **Reference**: Learn more about callback hell at callbackhell.com

---

API Calls with Callbacks

What are Callbacks?

Callbacks are functions passed as arguments to other functions that get executed later when an operation completes. In JavaScript, callbacks are commonly used for asynchronous operations.

Basic Example

Here's a simple example demonstrating the callback pattern:

```
// Function that uses a callback
function fetchUserData(userId, callback) {
  // Simulate an asynchronous operation (like an API call)
  setTimeout(() => {
    // Simulate success or error
    if (userId > 0) {
      // Success: call callback with null error and data
      callback(null, { id: userId, name: 'John Doe', email: 'john@example.com' });
    } else {
      // Error: call callback with error and null data
      callback(new Error('Invalid user ID'), null);
    }
  }, 1000); // Simulate 1 second delay
}

// Use the function with a callback
fetchUserData(123, function(error, userData) {
  // This function runs when the operation completes
  if (error) {
    // Handle error
    console.error('Error:', error);
  } else {
    // Handle success
    console.log('User data:', userData);
  }
});
```

How it works:

1. `fetchUserData` is called with a `userId` and a `callback` function
2. The function performs an asynchronous operation (simulated with `setTimeout`)
3. When the operation completes, it calls the `callback` function with the result
4. The callback receives two parameters: `(error, data)`
5. If successful: `callback(null, data)` - error is null, data contains the result
6. If failed: `callback(error, null)` - error contains the error, data is null

Key Points:

- Callbacks are functions passed to other functions
- The callback function always receives two parameters: `(error, data)`
- This pattern is called "error-first callbacks" - error always comes first
- The callback is executed when the asynchronous operation completes

The Problem: Callback Hell

When you have multiple nested callbacks, code becomes hard to read and maintain:

```
// ❌ BAD: Callback hell (nested callbacks)
fetchUserData(userId, function(error, user) {
  if (error) {
```

```
      console.error('Error fetching user:', error);
    } else {
      fetchUserPosts(user.id, function(error, posts) {
        if (error) {
          console.error('Error fetching posts:', error);
        } else {
          fetchPostComments(posts[0].id, function(error, comments) {
            if (error) {
              console.error('Error fetching comments:', error);
            } else {
              console.log('Comments:', comments);
              // More nested callbacks...
            }
          });
        }
      });
    }
  });
```

Problems with this approach:

- Hard to read (pyramid of doom)
- Difficult to maintain
- Error handling is scattered
- Difficult to debug

Following the callbackhell.com guidelines:

API Calls with Promises

What are Promises?

Promises represent a value that may be available now, or in the future, or never. They provide a cleaner way to handle asynchronous operations.

Basic Promise Example

```
// Function that returns a Promise
function fetchUserData(userId) {
  return fetch(`https://api.example.com/users/${userId}`)
    .then(response => {
      if (!response.ok) {
        throw new Error('Network response was not ok');
      }
      return response.json();
    });
}

// Using the Promise
fetchUserData(123)
  .then(userData => {
```

```
    console.log('User data:', userData);
  })
  .catch(error => {
    console.error('Error:', error);
  });
```

Chaining Promises

Promises can be chained to avoid callback hell:

```
// ✅ GOOD: Promise chaining (no nesting)
fetchUserData(userId)
  .then(user => {
    console.log('User:', user);
    return fetchUserPosts(user.id);
  })
  .then(posts => {
    console.log('Posts:', posts);
    return fetchPostComments(posts[0].id);
  })
  .then(comments => {
    console.log('Comments:', comments);
  })
  .catch(error => {
    console.error('Error in chain:', error);
  });
```

Benefits:

- Flatter code structure
- Single error handler for the entire chain
- More readable than nested callbacks
- Better error propagation

Promise Example in React Native

```
import { useState } from 'react';

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useState(() => {
    fetchUserData(userId)
      .then(userData => {
        setUser(userData);
        setLoading(false);
      })
```

```
        .catch(err => {
          setError(err.message);
          setLoading(false);
        });
    }, [userId]);

    if (loading) return <Text>Loading...</Text>;
    if (error) return <Text>Error: {error}</Text>;
    return <Text>{user.name}</Text>;
  }
```

Modern Way: Async/Await

What is Async/Await?

`async/await` is syntactic sugar built on top of Promises. It makes asynchronous code look and behave more like synchronous code, making it easier to read and write.

Basic Async/Await Example

```
// Function marked with 'async' returns a Promise
async function fetchUserData(userId) {
  const response = await fetch(`https://api.example.com/users/${userId}`);
  if (!response.ok) {
    throw new Error('Network response was not ok');
  }
  const userData = await response.json();
  return userData;
}

// Using async/await
async function displayUser(userId) {
  try {
    const userData = await fetchUserData(userId);
    console.log('User data:', userData);
  } catch (error) {
    console.error('Error:', error);
  }
}
```

Chaining with Async/Await

```
// ✅ BEST: Async/await (reads top-to-bottom)
async function loadUserData(userId) {
  try {
    const user = await fetchUserData(userId);
    console.log('User:', user);
```

```
    const posts = await fetchUserPosts(user.id);
    console.log('Posts:', posts);

    const comments = await fetchPostComments(posts[0].id);
    console.log('Comments:', comments);
  } catch (error) {
    console.error('Error:', error);
  }
}
```

Benefits:

- Reads like synchronous code (top-to-bottom)
- Easier to understand for beginners
- Better error handling with try/catch
- Less boilerplate than Promises

Async/Await in React Native Components

```
import { useState, useEffect } from 'react';

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    async function loadUser() {
      try {
        setLoading(true);
        const userData = await fetchUserData(userId);
        setUser(userData);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    }

    loadUser();
  }, [userId]);

  if (loading) return <Text>Loading...</Text>;
  if (error) return <Text>Error: {error}</Text>;
  return <Text>{user.name}</Text>;
}
```

Error Handling with Try/Catch

Why Error Handling is Important

Asynchronous operations can fail for many reasons:

- Network errors
- Server errors
- Invalid responses
- Timeouts

Always handle errors! Unhandled errors can crash your app or leave it in a bad state.

Try/Catch with Async/Await

```
async function fetchUserData(userId) {
  try {
    const response = await fetch(`https://api.example.com/users/${userId}`);

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const userData = await response.json();
    return userData;
  } catch (error) {
    // Handle different types of errors
    if (error.name === 'TypeError') {
      console.error('Network error:', error.message);
    } else if (error.name === 'SyntaxError') {
      console.error('Invalid JSON response:', error.message);
    } else {
      console.error('Unknown error:', error.message);
    }
    throw error; // Re-throw to let caller handle it
  }
}
```

Error Handling in React Components

```
import { useState } from 'react';

function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);

  const handleLogin = async () => {
    try {
      setLoading(true);
      setError(null); // Clear previous errors
```

```
      const result = await loginUser(email, password);

      if (result.success) {
        // Handle success
        console.log('Login successful');
      } else {
        // Handle API error response
        setError(result.message);
      }
    } catch (error) {
      // Handle network/other errors
      console.error('Login error:', error);
      setError('Network error. Please check your connection.');
    } finally {
      setLoading(false);
    }
  };

  return (
    <View>
      {error && <Text style={{ color: 'red' }}>{error}</Text>}
      <TextInput value={email} onChangeText={setEmail} />
      <TextInput value={password} onChangeText={setPassword} secureTextEntry />
      <Button
        title={loading ? 'Logging in...' : 'Login'}
        onPress={handleLogin}
        disabled={loading}
      />
    </View>
  );
}
```

Best Practices for Error Handling

1. **Always use try/catch with async/await**
2. **Handle errors at the right level** - Don't catch and ignore
3. **Provide user-friendly error messages**
4. **Log errors for debugging**
5. **Use finally block for cleanup**

```
async function loadData() {
  let data = null;
  try {
    data = await fetchData();
    // Process data
  } catch (error) {
    // Handle error
    console.error('Error loading data:', error);
    showErrorToUser('Failed to load data. Please try again.');
  } finally {
```

```
    // Always runs, regardless of success or failure
    // Good for cleanup (e.g., hiding loading spinner)
    setLoading(false);
  }
}
```

Best Practices

1. Always Handle Errors

```
// ❌ BAD: No error handling
async function badExample() {
  const data = await fetchData();
  console.log(data);
}

// ✅ GOOD: Proper error handling
async function goodExample() {
  try {
    const data = await fetchData();
    console.log(data);
  } catch (error) {
    console.error('Error:', error);
    // Handle error appropriately
  }
}
```

2. Use Async/Await for Readability

```
// ❌ BAD: Promise chains can get messy
fetchData()
  .then(data => processData(data))
  .then(processed => saveData(processed))
  .then(saved => displayData(saved))
  .catch(error => handleError(error));

// ✅ GOOD: Async/await is cleaner
async function handleData() {
  try {
    const data = await fetchData();
    const processed = await processData(data);
    const saved = await saveData(processed);
    displayData(saved);
  } catch (error) {
    handleError(error);
  }
}
```

3. Keep Functions Focused

```
// ❌ BAD: Function does too much
async function doEverything() {
  const user = await fetchUser();
  const posts = await fetchPosts();
  const comments = await fetchComments();
  // ... 50 more lines
}

// ✅ GOOD: Small, focused functions
async function loadUserData() {
  const user = await fetchUser();
  return user;
}

async function loadPosts(userId) {
  const posts = await fetchPosts(userId);
  return posts;
}
```

4. Handle Loading States

```
async function loadData() {
  setLoading(true);
  try {
    const data = await fetchData();
    setData(data);
  } catch (error) {
    setError(error.message);
  } finally {
    setLoading(false); // Always stop loading
  }
}
```

Real Examples from This Project

Example 1: Login Service (Async/Await Pattern)

File: `services/authService.js`

```
export const loginUser = async (email, password) => {
  try {
    const url = `${API_BASE_URL}/auth/login`;
    console.log('Login API URL:', url);

    const response = await fetch(url, {
```

```
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({ email, password }),
    });

    const data = await response.json();

    if (response.ok && data.success) {
      return {
        success: true,
        data: data.data,
        message: data.message,
      };
    } else {
      return {
        success: false,
        message: data.message || 'Login failed. Please try again.',
      };
    }
  } catch (error) {
    console.error('Login API error:', error);
    return {
      success: false,
      message: 'Network error. Please check your connection and try again.',
    };
  }
};
```

Key Points:

- Uses `async/await` for clean code
- Try/catch for error handling
- Returns consistent result format
- Handles both API errors and network errors

Example 2: Using the Service in a Component

File: `Screens/LoginScreen.js`

```
const handleSubmit = async () => {
  // Validation
  if (!emailInput || !password) {
    alert('Error', 'Please fill in all fields');
    return;
  }

  setLoading(true);
  const result = await loginUser(emailInput, password);
```

```
    if (result.success) {
      login(result.data.user, result.data.token);
      alert('Success', `Welcome, ${result.data.user.name || emailInput}!`);
      navigation.navigate('MainTabs');
      setEmailInput('');
      setPassword('');
    } else {
      alert('Error', result.message);
    }

    setLoading(false);
  };
```

Key Points:

- Async function for API call
- Loading state management
- Error handling through result object
- Clean user feedback

Example 3: Profile Loading with Error Handling

File: `Screens/ProfileScreen.js`

```
useEffect(() => {
  const loadProfile = async () => {
    if (!token) {
      setError('No authentication token found');
      setLoading(false);
      return;
    }

    setLoading(true);
    setError(null);

    const result = await getUserProfile(token);

    if (result.success) {
      setProfile(result.data);
    } else {
      setError(result.message);
    }

    setLoading(false);
  };

  loadProfile();
}, [token]);
```

Key Points:

- Async function inside useEffect
- Proper loading and error states
- Clean error messages for users

Summary

Evolution of Async Code

1. **Callbacks** → Can lead to callback hell, but can be managed with named functions
2. **Promises** → Better than callbacks, but chains can get long
3. **Async/Await** → Modern, readable, easy to write and maintain

Key Takeaways

✅ **Always use async/await** for new code
✅ **Always handle errors** with try/catch
✅ **Always manage loading states**
✅ **Keep functions focused** and small
✅ **Return consistent formats** from API functions
✅ **Provide user-friendly error messages**

Resources

- Callback Hell Guide - Understanding and avoiding callback hell
- MDN: Async/Await
- MDN: Promises
- JavaScript.info: Promises

Happy Coding! 🚀